

Tarea 3

Nombre y apellido: Nicolas Seguel Número de lista: 49 Puntaje total: 23

En esta tarea trabajará con una señal de audio real y analizará cómo se transforma al pasar por un sistema LTI. Comience por obtener o grabar una señal de audio mono.

4

1. [4p] *Sistema y señales.* Cargue la señal, que tenga al menos $N = 50000$ muestras y normalice su media y varianza para que sean $\mu = 0$ y $\sigma^2 = 1$. Llame a esta señal $f[n]$.

Encuentre la salida $g[n] = h[n] * f[n]$ cuando el sistema tiene una respuesta al impulso

$$h[n] = (1 - \alpha) \alpha^n \Gamma[n], \quad 0 < \alpha < 1.$$

Use $\alpha = 0,8$. Para implementar la convolución con longitud finita, trunque $h[n]$ a $0 \leq n \leq L_h$ con $L_h = 200$ (justifique que el error de truncamiento es pequeño). Puede usar directamente la convolución o vía FFT.

Para evitar efectos de borde, descarte todas las muestras necesarias al comienzo y al final de $g[n]$ para que el largo sea el mismo que el de $f[n]$.

Escuche el resultado $g[n]$.

8

2. [8p] *Autocorrelaciones.*

4

- a. [4p] Estime y grafique la autocorrelación de entrada $\hat{r}_f[l]$ y de salida $\hat{r}_g[l]$ para retardos $l \in \{-L, \dots, L\}$ con $L = 500$, tanto en su versión sesgada como no sesgada:

$$\hat{r}_x^{(\text{bias})}[l] = \frac{1}{N} \sum_n x[n] x^*[n-l], \quad \hat{r}_x^{(\text{unb})}[l] = \frac{1}{N-|l|} \sum_n x[n] x^*[n-l].$$

Mencione las diferencias entre ambas versiones.

4

- b. [4p] Compare la autocorrelación calculada $\hat{r}_g[l]$ con la autocorrelación predicha por

$$\hat{r}_g^{(\text{pred})}[l] = h[l] * h^*[l] * f[n].$$

y encuentre el error relativo entre ellas

$$\varepsilon = \frac{\|\hat{r}_g^{(\text{pred})} - \hat{r}_g\|_2}{\|\hat{r}_g\|_2}.$$

8

3. [8p] *Correlación cruzada.* Análogamente al problema anterior,

4

- a. [4p] estime y grafique la correlación cruzada entrada-salida

$$\hat{r}_{fg}[l] = \frac{1}{N-|l|} \sum_n f[n] g^*[n-l]$$

4

b. [4p] y compárela con la correlación cruzada predicha

$$\hat{r}_{fg}^{(\text{pred})}[l] = h^*[-l] * \hat{r}_f[l]$$

y encuentre el error relativo entre ellas.

3

4. [4p] *Discusión.*

1

a. [2p] Explique cómo el parámetro α controla el “suavizado” temporal y cómo esto se refleja en $r_g[l]$ y $r_{fg}[l]$. Repita brevemente con $\alpha \in \{0,5,0,9\}$ (sin rehacer todo el informe; añada una figura comparativa). Escuche los resultados para diferentes α .

2

b. [2p] Comente el efecto para los valores de L_h y L .

IEE3584 - Procesamiento Avanzado de Señales

Tarea 3

Nicolás Seguel

Agosto 2025

Problema 1

Se carga el archivo de audio `audio.wav`. Luego se normaliza la señal y se convoluciona con $h[n]$. Como la señal exponencial decae rápidamente, se utiliza un filtro de longitud $L_h = 200$, donde el valor de $h[n]$ se vuelve despreciable, por lo que es una buena aproximación. Finalmente, se guarda la señal resultante en `output.wav`.

Listing 1: Código para el Problema 1

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.io import wavfile
4 import os
5
6 audio_path = os.path.join('PAS', 'tarea3', 'data', 'audio.wav')
7 fs, data = wavfile.read(audio_path)
8
9 if data.ndim > 1:
10     data = data[:, 0]
11
12 # Normalizar la señal
13 data = data.astype(np.float64)
14 data = (data - np.mean(data)) / np.std(data)
15
16 # Definir h(n)
17 a = 0.8
18 N = 200
19 n = np.arange(N)
20 h = (1 - a) * (a ** n)
21
22 # Convolución con FFT
23 L = len(data) + len(h) - 1
24 data_fft = np.fft.fft(data, n=L)
25 h_fft = np.fft.fft(h, n=L)
26 y_fft = data_fft * h_fft
27 y = np.fft.ifft(y_fft).real
28 y = y[:len(data)]
29 t_y = np.arange(len(y)) / fs
30
31 # Reproducir la salida
32 output_path = os.path.join('PAS', 'tarea3', 'data', 'output.wav')
```

```
33 wavfile.write(output_path, fs, y.astype(np.float32))
```

Problema 2

Se estiman las autocorrelaciones sesgada y no sesgada para las señales de entrada y salida, con retardos $l \in \{-500, \dots, 500\}$. Además, se compara la autocorrelación estimada de salida $\hat{r}_g[l]$ con la predicha, y se obtiene un error relativo de 0,11.

Listing 2: Código para el Problema 2

```
1 import numpy as np
2 from scipy.io import wavfile
3 import matplotlib.pyplot as plt
4
5 def autocorrelacion_fft(x, rango, sesgada=True):
6     N = len(x)
7     # FFT y cálculo de autocorrelación
8     Xf = np.fft.fft(x, n=2*N)
9     r = np.fft.ifft(np.abs(Xf)**2).real[:N]
10    r = np.concatenate((r[::-1][:-1], r))
11
12    mid = len(r) // 2
13    r = r[mid + rango[0]: mid + rango[-1] + 1]
14
15    if not sesgada:
16        lag = np.arange(rango[0], rango[-1] + 1)
17        r = r / (N - np.abs(lag))
18    else:
19        r = r / N
20    return r
21
22 if __name__ == "__main__":
23     # Cargar audios
24     fs1, x1 = wavfile.read('PAS\\tarea3\\data\\audio.wav')
25     fs2, x2 = wavfile.read('PAS\\tarea3\\data\\output.wav')
26
27     if x1.ndim > 1:
28         x1 = x1[:, 0]
29     if x2.ndim > 1:
30         x2 = x2[:, 0]
31
32     x1 = x1[:100000]
33     x2 = x2[:100000]
34
35     x1 = (x1 - np.mean(x1)) / np.std(x1)
36     x2 = (x2 - np.mean(x2)) / np.std(x2)
37
38     rango = np.arange(-500, 501)
39
40     auto_sesgada_1 = autocorrelacion_fft(x1, rango, sesgada=True)
41     auto_no_sesgada_1 = autocorrelacion_fft(x1, rango, sesgada=False)
42     auto_sesgada_2 = autocorrelacion_fft(x2, rango, sesgada=True)
43     auto_no_sesgada_2 = autocorrelacion_fft(x2, rango, sesgada=False)
```

```

44
45 # Autocorrelación predicha
46 alpha = 0.8
47 Lh = 200
48 h = (1 - alpha) * alpha**np.arange(Lh+1)
49
50 # Predicción: h * h* * rf
51 rh = np.convolve(h, h[::-1], mode='full')
52 rg_pred = np.convolve(rh, auto_sesgada_1, mode='full')
53 rg_est = auto_no_sesgada_2
54 rg_pred = rg_pred[len(rg_pred)//2 + rango[0] : len(rg_pred)//2 + rango
55 [-1] + 1]
56 error_rel = np.linalg.norm(rg_pred - rg_est) / np.linalg.norm(rg_est)
57 print(f"Error relativo: {error_rel:.4f}")
58
59 # Gráfico 1: Sesgada vs No Sesgada
60 plt.figure(figsize=(10,8))
61
62 plt.subplot(2,1,1)
63 plt.plot(rango, auto_sesgada_1, label='Sesgada audio.wav')
64 plt.plot(rango, auto_no_sesgada_1, label='No Sesgada audio.wav',
65          linestyle="--")
66 plt.title('Autocorrelación de audio.wav')
67 plt.xlabel('Desplazamiento (muestras)')
68 plt.ylabel('Autocorrelación')
69 plt.legend(); plt.grid()
70
71 plt.subplot(2,1,2)
72 plt.plot(rango, auto_sesgada_2, label='Sesgada output.wav')
73 plt.plot(rango, auto_no_sesgada_2, label='No Sesgada output.wav',
74          linestyle="--")
75 plt.title('Autocorrelación de output.wav')
76 plt.xlabel('Desplazamiento (muestras)')
77 plt.ylabel('Autocorrelación')
78 plt.legend(); plt.grid()
79
80 plt.tight_layout()
81 plt.savefig('PAS\\tarea3\\data\\autocorrelacion_sesgada_vs_nosesgada.
82 png')
83 plt.show()
84
85 # Gráfico 2: Estimada vs Predicha
86 plt.figure(figsize=(8,6))
87 plt.plot(rango, rg_est, label='Estimación Sesgada')
88 plt.plot(rango, rg_pred, label='Predicha', linestyle="--")
89 plt.title('Autocorrelación de output.wav (estimada vs predicha)')
90 plt.xlabel('Desplazamiento (muestras)')
91 plt.ylabel('Autocorrelación')
92 plt.legend(); plt.grid()
93
94 plt.tight_layout()
95 plt.savefig('PAS\\tarea3\\data\\autocorrelacion_predicha.png')
96 plt.show()

```

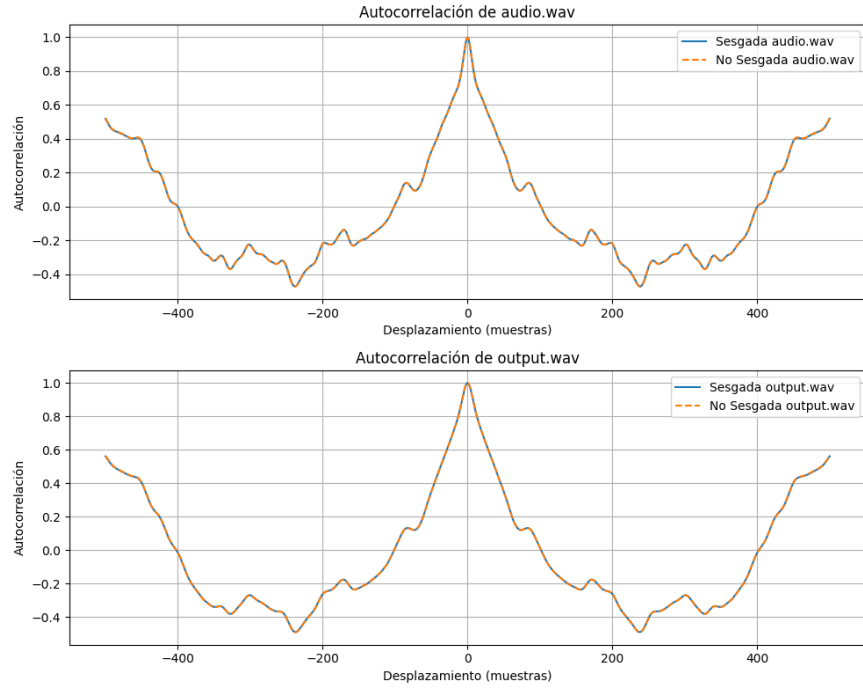


Figura 1: Autocorrelación sesgada y no sesgada de las señales de entrada y salida.

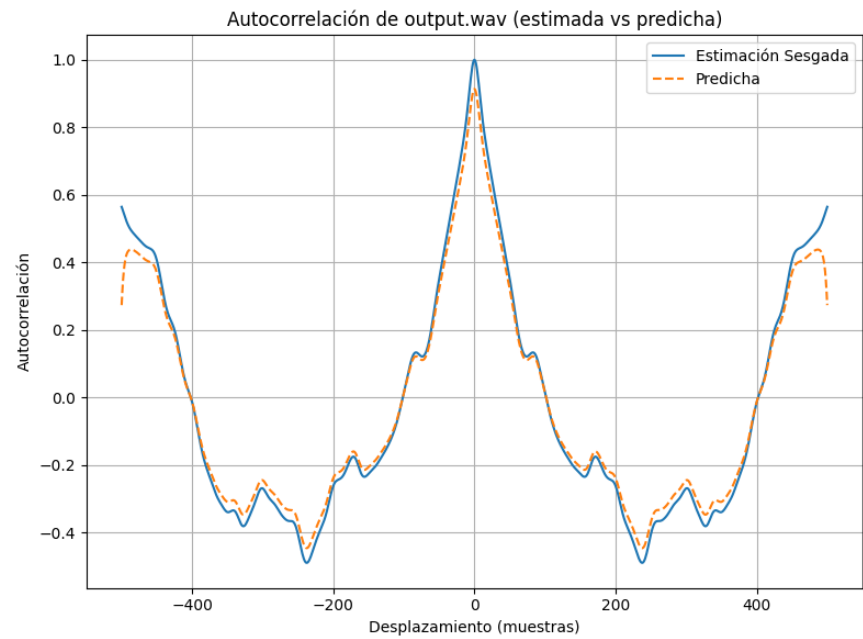


Figura 2: Comparación entre autocorrelación estimada y predicha de la señal de salida.

Problema 3

Se calcula la correlación cruzada entrada-salida tanto sesgada como no sesgada. Luego, se compara con la correlación cruzada predicha:

$$\hat{r}_{fg}^{(pred)}[l] = h^*[-l] * \hat{r}_f[l],$$

y se obtiene un error relativo igual a 0,1333.

Listing 3: Código para el Problema 3

```
1 import numpy as np
2 from scipy.io import wavfile
3 import matplotlib.pyplot as plt
4 from p2 import autocorrelacion_fft
5
6 def correlacion_cruzada_fft(x, y, rango, sesgada=True):
7     """
8     Correlación cruzada de x e y
9     """
10    N = len(x)
11    Xf = np.fft.fft(x)
12    Yf = np.fft.fft(y)
13    r = np.fft.ifft(Xf * np.conj(Yf)).real[:N] # correlación cruzada
14    r = np.concatenate((r[::-1][:-1], r))
15
16    mid = len(r) // 2
17    r = r[mid + rango[0]: mid + rango[-1] + 1]
18
19    if not sesgada:
20        lag = np.arange(rango[0], rango[-1] + 1)
21        r = r / (N - np.abs(lag))
22    else:
23        r = r / N
24    return r
25
26 # Cargar audios
27 fs1, x1 = wavfile.read('PAS\\tarea3\\data\\audio.wav')
28 fs2, x2 = wavfile.read('PAS\\tarea3\\data\\output.wav')
29
30 if x1.ndim > 1:
31     x1 = x1[:, 0]
32 if x2.ndim > 1:
33     x2 = x2[:, 0]
34
35 x1 = x1[:100000]
36 x2 = x2[:100000]
37
38 x1 = (x1 - np.mean(x1)) / np.std(x1)
39 x2 = (x2 - np.mean(x2)) / np.std(x2)
40
41 # Correlaciones cruzadas
42 rango = np.arange(-500, 501)
43
44 cross_sesgada = correlacion_cruzada_fft(x1, x2, rango, sesgada=True)
```

```

45 cross_no_sesgada = correlacion_cruzada_fft(x1, x2, rango, sesgada=False)
46
47
48 # Parte 3b: Correlación cruzada predicha
49 alpha = 0.8
50 Lh = 200
51 h = (1 - alpha) * alpha*np.arange(Lh+1)
52
53 rf = autocorrelacion_fft(x1, rango, sesgada=True)
54 h_conj = h[::-1]
55 rfg_pred = np.convolve(h_conj, rf, mode="full")
56 mid = len(rf) // 2 + len(h) - 1
57 rfg_pred = rfg_pred[mid + rango[0] : mid + rango[-1] + 1]
58
59 # Error relativo
60 error_rel = np.linalg.norm(rfg_pred - cross_sesgada) / np.linalg.norm(
    cross_sesgada)
61 print(f"Error relativo parte 3b: {error_rel:.4f}")
62
63
64 # Gráfico 1: Sesgada vs No Sesgada
65 plt.figure(figsize=(10,6))
66 plt.plot(rango, cross_sesgada, label='Cruzada Sesgada f-g')
67 plt.plot(rango, cross_no_sesgada, label='Cruzada No Sesgada f-g',
    linestyle="--")
68 plt.title('Correlación Cruzada entrada-salida')
69 plt.xlabel('Desplazamiento (muestras)')
70 plt.ylabel('Correlación cruzada')
71 plt.legend(); plt.grid()
72
73 plt.tight_layout()
74 plt.savefig('PAS\\tarea3\\data\\correlacion_cruzada.png')
75 plt.show()
76
77
78 # Gráfico 2: Estimada vs Predicha (parte 3b)
79 plt.figure(figsize=(8,6))
80 plt.plot(rango, cross_sesgada, label='Estimación Sesgada')
81 plt.plot(rango, rfg_pred, label='Predicha', linestyle="--")
82 plt.title('Correlación Cruzada (estimada vs predicha)')
83 plt.xlabel('Desplazamiento (muestras)')
84 plt.ylabel('Correlación cruzada')
85 plt.legend(); plt.grid()
86
87 plt.tight_layout()
88 plt.savefig('PAS\\tarea3\\data\\correlacion_cruzada_predicha.png')
89 plt.show()

```

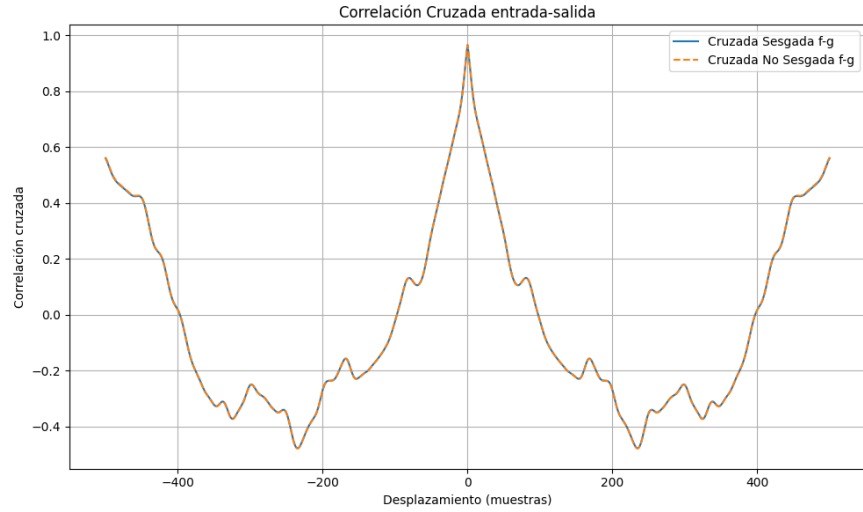



Figura 3: Correlación cruzada sesgada y no sesgada entre entrada y salida.

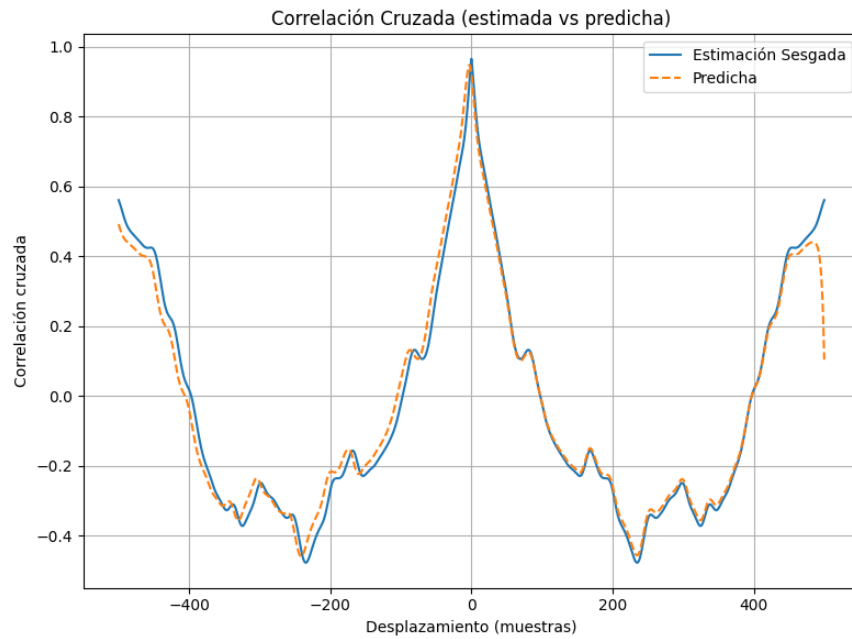


Figura 4: Comparación entre correlación cruzada estimada y predicha.

Problema 4

Parte a

El filtro $h[n]$ es un filtro IIR causal de tipo exponencial. Actúa como un filtro pasa bajos, por lo que suaviza la señal de entrada. En un audio se traduce en una reducción del ruido de alta frecuencia, pero también puede causar una pérdida de detalles en la señal. El parámetro α controla la rapidez con la que el filtro atenúa las frecuencias altas: valores cercanos a 1 resultan en una mayor atenuación, mientras que valores cercanos a 0 permiten pasar más frecuencias altas.

El efecto del filtro en $r_g[l]$ y $r_{fg}[l]$ es que ambos se ven suavizados y atenuados en comparación con $r_f[l]$. Esto se debe a que la convolución con el filtro introduce una dependencia temporal, lo que reduce la variabilidad de la señal y, por ende, sus correlaciones.

Parte b

El efecto de L_h en las estimaciones de autocorrelación y correlación cruzada es que un L_h más grande permite capturar más del efecto del filtro en la señal. Sin embargo, a partir de cierto valor, aumentar L_h tiene un impacto mínimo, ya que los valores de $h[n]$ se vuelven muy pequeños y contribuyen poco a la convolución.

Ahora bien, L determina la ventana de retardos para los cuales se calcula la autocorrelación y correlación cruzada. Un L más grande permite observar más detalles en las correlaciones a largo plazo, pero también más tiempo de cómputo y posible ruido en las estimaciones.